Estratégia Concursos

Aula 45

Polícia Federal (Agente de Polícia) Passo Estratégico de Informática - Prof Diego Carvalho

Autor:
Diego Carvalho

09 de Abril de 2025

98276239072 - FABIANO CARBONELL DA SILVA

Índice

1) O que é mais cobrado no assunto - Linguagem R - CEBRASPE 3

2) Roteiro de Revisão - Linguagem R 4

3) Aposta Estratégica - Linguagem R 12

4) Questões Estratégicas - Linguagem R - CEBRASPE 13

5) Questionário de Revisão - Linguagem R 23

6) Lista de Questões Estratégicas - Linguagem R - CEBRASPE 34

7) Gabarito de Questões Estratégicas - Linguagem R - CEBRASPE 39

8) Referências Bibliográficas - Linguagem R 40

98276239072 - FABIANO CARBONELL DA SILVA

O QUE É MAIS COBRADO DENTRO DO ASSUNTO?

Considerando os tópicos que compõem o nosso assunto, possuímos a seguinte distribuição percentual:

TÓPICO	% DE COBRANÇA [CEBRASPE]
Conceitos Básicos	02%
Operadores Suportados	04%
Variáveis	04%
Funções Genéricas	03%
Comentários	01%
Tipos de Dados	02%
Vetores	16%
Matrizes	17%
Frames	06%
Funções Estatísticas	08%
Principais Gráficos	10%
Diretórios e Bases de Dados	06%
Correlação e Regressão	05%
Scripts de Programação	10%
Funções Personalizadas	03%
Pacotes e Bibliotecas	03%

98276239072 - FABIANO CARBONELL DA SILVA

Roteiro de Revisão e Pontos do Assunto que Merecem Destaque

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

R é uma linguagem de programação e ambiente voltado para estatísticas, análise de dados e gráficos. R oferece uma vasta coleção de pacotes e bibliotecas para manipulação, modelagem e visualização de dados. Sua sintaxe permite fácil integração com outras linguagens, como C++ e Python, e é popular por suas capacidades de gráficos estatísticos e análise avançada. Vejamos os principais operadores:

OPERADORES ARITMÉTICOS	OPERADOR	DESCRIÇÃO	EXEMPLO
ADIÇÃO	+	Retorna a soma de valores.	Ex: 2+3 = 5
SUBTRAÇÃO	-	Retorna a subtração de valores.	Ex: 5-2 = 3
MULTIPLICAÇÃO	*	Retorna a multiplicação de valores.	Ex: 4*3 = 12
DIVISÃO	/	Retorna a divisão de valores.	Ex: 10/2 = 5
EXPONENCIAÇÃO	^ ou **	Retorna um valor à potência do outro.	Ex: 2^3 = 2**3 = 8
MÓDULO	%%	Retorna o resto da divisão de valores.	Ex: 10%%3 = 1
DIVISÃO INTEIRA	%/%	Retorna o quociente inteiro da divisão de valores.	Ex: 10%/%3 = 3

OPERADORES RELACIONAIS	OPERADOR	DESCRIÇÃO	EXEMPLO
MAIOR	>	Verifica se o primeiro valor é maior que o segundo.	Ex: 5>3 retorna TRUE
MENOR	<	Verifica se o primeiro valor é menor que o segundo.	Ex: 3<5 retorna TRUE
MAIOR OU IGUAL	>=	Verifica se o primeiro valor é maior ou igual ao segundo.	Ex: 6>=5 retorna FALSE
MENOR OU IGUAL	<=	Verifica se o primeiro valor é menor ou igual ao segundo.	Ex: 4<=3 retorna FALSE
IGUAL	==	Verifica se dois valores são iguais.	Ex: 3==3 retorna TRUE
DIFERENTE	!=	Verifica se dois valores são diferentes.	Ex: 3!=3 retorna FALSE

OPERADORES LÓGICOS	OPERADOR	DESCRIÇÃO	EXEMPLO
E	&	Retorna TRUE se ambos os operandos forem verdadeiros (avalia elemento a elemento).	Ex: TRUE & FALSE retorna FALSE
OU	|	Retorna TRUE se pelo menos um dos operandos for verdadeiro (avalia elemento a elemento).	Ex: TRUE | FALSE retorna TRUE
NÃO	!	Inverte o valor lógico de um operando.	Ex: !TRUE retorna FALSE
E (LÓGICO)	&&	Avalia apenas o primeiro elemento (E lógico para vetores inteiros).	Ex: TRUE && FALSE retorna FALSE

98276239072 - FABIANO CARBONELL DA SILVA

OU (LÓGICO)	||	Avalia apenas o primeiro elemento (OU lógico para vetores inteiros).	Ex: TRUE || FALSE retorna TRUE

OPERADORES DE ATRIBUIÇÃO	OPERADOR	DESCRIÇÃO	EXEMPLO
RECEBE	**<-** ou =	Atribui o valor do lado direito à variável do lado esquerdo.	Ex: x **<-** 5 ou x = 5 armazena 5 em x
IMPLICA	->	Atribui o valor do lado esquerdo à variável do lado direito.	Ex: 5 -> x armazena o valor 5 na variável x

Vejamos algumas funções matemáticas básicas:

FUNÇÕES MATEMÁTICAS	FUNÇÃO	DESCRIÇÃO	EXEMPLO
RAIZ QUADRADA	sqrt()	Calcula a raiz quadrada de um número.	Ex: sqrt(16) retorna 4
LOGARITMO	log()	Calcula o logaritmo natural de um número.	Ex: log(10) retorna ≈ 2.302585
VALOR ABSOLUTO	abs()	Retorna o valor absoluto (positivo) de um número.	Ex: abs(-5) retorna 5

Vejamos as regras básicas de nomenclatura de variáveis:

REGRAS BÁSICAS	DESCRIÇÃO	EXEMPLO
CARACTERES PERMITIDOS	Os nomes de variáveis podem conter letras, números, pontos (.) e sublinhados (_).	Ex: minha_variavel, minha.variavel, variavel_1.
PRIMEIRO CARACTERE	O nome de uma variável deve começar com uma letra ou um ponto seguido por uma letra. Não pode começar com um número.	Ex: .minhaVariavel (válido), 1variavel (inválido).
CASE-SENSITIVE	A linguagem R diferencia maiúsculas de minúsculas.	Ex: minhaVariavel e minhavariavel são variáveis diferentes.
PALAVRAS RESERVADAS	Não se pode utilizar palavras reservadas da linguagem R como nomes de variáveis.	Ex: if, else, TRUE, FALSE, for, etc.

NOME DA VARIÁVEL	VÁLIDO?	EXPLICAÇÃO
nota1	Sim	Começa com uma letra, contém números.
1nota	Não	Não pode começar com número.
Nota1	Sim	Válido, mas diferente de nota1.
nota_1	Sim	Contém sublinhado, válido.
nota.1	Sim	Contém ponto, válido.

É importante estudar também o escopo de variáveis:

VARIÁVEIS	DESCRIÇÃO
GLOBAIS	Uma variável global é aquela que é definida fora de qualquer função e pode ser acessada e modificada em qualquer lugar do código, incluindo dentro de funções, a menos que uma

98276239072 - FABIANO CARBONELL DA SILVA

	variável local com o mesmo nome seja criada dentro da função. Elas têm um escopo global, ou seja, podem ser utilizadas em qualquer parte do programa depois de serem definidas.
LOCAIS	Uma variável local é aquela que é definida dentro de uma função e só pode ser acessada dentro dessa função. Essas variáveis têm um escopo local, o que significa que deixam de existir assim que a execução da função termina. Elas não podem ser acessadas ou modificadas fora da função onde foram criadas.

Vejamos agora algumas funções genéricas muito utilizadas:

FUNÇÕES GENÉRICAS	DESCRIÇÃO
print()	Essa função exibe no console o valor de um objeto ou resultado de uma expressão. Em R, a função print() é amplamente utilizada para retornar ou visualizar dados e resultados intermediários durante a execução de um script, sendo uma função essencial para depuração e acompanhamento de código.
ls()	Essa função retorna uma lista com o nome dos objetos (variáveis, funções, etc.) que estão atualmente disponíveis no ambiente de trabalho (workspace) do R. Ela é útil para visualizar quais objetos estão carregados na memória e identificar quais dados ou funções estão prontos para serem manipulados.
xor()	Essa função realiza uma operação lógica "OU exclusivo" (exclusive OR) entre dois valores lógicos. O resultado será verdadeiro (TRUE) se e somente se um dos valores for verdadeiro, mas não ambos. Ou seja, retorna TRUE quando os dois argumentos são diferentes (um é TRUE e o outro é FALSE), e FALSE quando são iguais.
class()	Essa função retorna a classe de um objeto em R, como "numeric", "character", "factor", "matrix", etc. Isso é útil para entender o tipo de dado com o qual está lidando e garantir que funções e operações apropriadas sejam aplicadas. Identificar a classe é essencial para manipulação adequada de objetos.

Em R, vetores são estruturas de dados unidimensionais que armazenam elementos do mesmo tipo (numérico, lógico ou caractere). Operações matemáticas e funções são aplicadas de forma vetorizada, ou seja, executadas simultaneamente em todos os elementos. R trata automaticamente a indexação de vetores, facilitando o acesso e a modificação de seus valores. Vejamos as principais funções de vetores:

FUNÇÕES DE VETORES	DESCRIÇÃO
c()	Essa função combina valores para criar um vetor. Ela é uma das funções mais usadas em R, permitindo concatenar elementos de qualquer tipo (números, caracteres, lógicos, etc.) em uma estrutura unidimensional. É a base para construir vetores, listas e outras estruturas de dados em R.
seq()	Essa função gera uma sequência de números, com valores inicial e final definidos, podendo controlar o intervalo entre eles. Ela permite especificar o comprimento da sequência (length.out) ou o incremento entre os números (by), facilitando a criação de séries numéricas ou temporais.
rep()	Essa função repete elementos de um vetor um número especificado de vezes. Útil para replicar valores constantes ou gerar sequências com padrões definidos, essa função permite controlar quantas vezes cada elemento ou o vetor completo será repetido.
sample()	Essa função cria uma amostra aleatória de um conjunto de dados, com ou sem reposição. É amplamente usada em simulações e experimentos de Monte Carlo, onde a randomização é essencial. O número de amostras pode ser ajustado e permite-se definir se a amostragem será com reposição.

98276239072 - FABIANO CARBONELL DA SILVA

set.seed()	Essa função define a semente do gerador de números aleatórios, garantindo reprodutibilidade em amostragens ou simulações aleatórias. Ao definir uma semente, você assegura que as mesmas sequências aleatórias sejam geradas em futuras execuções do código.
rev()	Essa função inverte a ordem dos elementos de um vetor. Se aplicado em vetores numéricos, lógicos ou de caracteres, a sequência dos elementos será revertida, retornando o vetor de trás para frente. É útil para manipulação e organização de dados em sentido inverso.
sort()	Essa função ordena os elementos de um vetor em ordem crescente por padrão, mas também pode ordenar em ordem decrescente. Ela é usada para organizar dados numéricos, lógicos ou de caracteres, permitindo análises mais fáceis e melhor visualização das distribuições de valores.

Em R, matrizes são estruturas de dados bidimensionais que armazenam elementos do mesmo tipo (numérico, lógico ou caractere) organizados em linhas e colunas. As operações matemáticas em matrizes são aplicadas de forma elementar ou algébrica, facilitando cálculos matriciais. Matrizes são amplamente usadas em álgebra linear e análise de dados em R. Vejamos a seguir as principais funções de matrizes:

FUNÇÕES DE MATRIZES	DESCRIÇÃO
cbind()	Essa função combina vetores, matrizes ou data frames lado a lado, ou seja, por colunas. Cada argumento passado para a função é tratado como uma nova coluna, e as linhas são alinhadas de acordo com a ordem original. Útil para juntar dados de forma matricial, respeitando as dimensões das colunas.
rbind()	Essa função combina vetores, matrizes ou data frames por linhas. Cada argumento é tratado como uma nova linha, e as colunas são alinhadas na mesma ordem. Ideal para anexar observações ou dados de diferentes conjuntos em uma estrutura unificada de linhas.
dim()	Essa função retorna ou define as dimensões de um objeto, como matrizes ou arrays, em R. Ela fornece um vetor com o número de linhas e colunas (ou dimensões adicionais) da estrutura. Quando usada com atribuição, pode modificar a forma de um objeto, convertendo, por exemplo, um vetor em matriz.
matrix()	Essa função cria uma matriz em R a partir de um vetor ou de uma sequência de números. O usuário pode especificar o número de linhas e colunas, preenchendo a matriz por linha ou por coluna. É fundamental para trabalhar com dados organizados em duas dimensões em R.
t()	Essa função realiza a transposição de uma matriz ou data frame, trocando suas linhas por colunas e vice-versa. É útil em diversas operações matemáticas e na manipulação de dados, onde a estrutura dos dados precisa ser reorganizada para facilitar a análise.
diag()	Essa função tem dois usos principais: criar uma matriz diagonal com os valores fornecidos ou extrair a diagonal de uma matriz existente. É uma função chave em álgebra linear para gerar matrizes de identidade ou acessar os elementos principais de uma matriz quadrada.
det()	Essa função calcula o determinante de uma matriz quadrada. O determinante é um valor escalar que resume certas propriedades da matriz, como a invertibilidade. Em álgebra linear, é crucial para resolver sistemas lineares e entender transformações lineares.

Data Frames são estruturas de dados bidimensionais que permitem armazenar diferentes tipos de dados (numéricos, caracteres, fatores, etc.) em colunas. São amplamente utilizados para análise de dados, pois cada coluna pode representar uma variável, e cada linha, uma observação.

98276239072 - FABIANO CARBONELL DA SILVA

Criados pela função data.frame(), Data Frames permitem fácil manipulação e filtragem de dados, sendo comparáveis a tabelas em bancos de dados ou planilhas. Vejamos as principais funções:

FUNÇÕES DE DATA FRAMES	DESCRIÇÃO
data.frame()	Essa função cria estruturas de dados bidimensionais em R, organizadas em linhas e colunas, onde cada coluna pode conter diferentes tipos de dados (numérico, caracter, lógico, etc.). É uma das principais estruturas de dados em R para armazenar e manipular grandes volumes de informações tabulares.
order()	Essa função retorna a ordem de classificação dos elementos de um vetor ou data frame. Ela não classifica diretamente, mas retorna um índice de posição para que, ao aplicar esse índice, seja possível obter a sequência ordenada dos elementos. É útil para ordenar dados complexos com base em várias colunas.
aggregate()	Essa função é usada para aplicar funções estatísticas (como sum, mean, etc.) a subconjuntos de dados agrupados em torno de uma ou mais variáveis de agrupamento. Comumente usada em análise de dados para realizar cálculos estatísticos agrupados (ex.: somar ou calcular média por grupos específicos).

Agora vamos ver algumas funções estatísticas básicas:

FUNÇÕES ESTATÍSTICAS	DESCRIÇÃO
sum()	Essa função calcula a soma total de todos os elementos em um vetor ou coluna de um data frame. É amplamente usada para obter o total de valores numéricos em um conjunto de dados. Pode ser aplicada em vetores de números inteiros, reais ou complexos.
mean()	Essa função calcula a média aritmética de um conjunto de dados, somando todos os elementos de um vetor e dividindo pelo número total de elementos. É usada para obter uma medida central dos dados.
median()	Essa função calcula a mediana de um conjunto de dados, que é o valor central que separa a metade inferior da metade superior dos valores. Se houver um número par de valores, ela retorna a média dos dois valores centrais.
min()	Essa função retorna o menor valor em um vetor ou conjunto de dados. Ela é usada para identificar o valor mínimo dentro de uma sequência numérica.
max()	Essa função retorna o maior valor dentro de um vetor ou conjunto de dados. Essa função é utilizada para determinar o valor máximo em um conjunto numérico.
var()	Essa função calcula a variância amostral de um vetor numérico, que mede a dispersão dos dados em relação à média. Ela é útil para entender como os dados variam em relação ao valor central.
sd()	Essa função calcula o desvio padrão de um conjunto de dados numéricos, que indica quanto os valores se desviam da média. O desvio padrão é a raiz quadrada da variância e é usado para medir a variabilidade dos dados.

R é amplamente conhecido por suas poderosas capacidades gráficas. Ele oferece funções nativas para criar gráficos de linhas, barras, histogramas, entre outros. A linguagem também suporta

98276239072 - FABIANO CARBONELL DA SILVA

pacotes como ggplot2, que permite a criação de gráficos complexos e personalizáveis com base na gramática dos gráficos. Esses recursos fazem do R uma excelente ferramenta para visualização de dados, permitindo análises visuais detalhadas. Vejamos as principais funções:

FUNÇÕES DE GRÁFICOS	DESCRIÇÃO
barplot()	Essa função cria gráficos de barras verticais ou horizontais. É usada para representar visualmente os valores numéricos associados a categorias. As alturas ou comprimentos das barras são proporcionais aos valores representados, permitindo a fácil comparação entre as categorias.
pie()	Essa função gera gráficos de pizza (setores), que são usados para exibir a distribuição proporcional de um conjunto de dados categorizados. Cada fatia do gráfico representa uma parte do todo, com o tamanho da fatia proporcional ao valor correspondente.
plot()	Essa função é a função genérica para criar gráficos de dispersão em R. Ela é usada para gerar visualizações gráficas de dados, como relações entre variáveis, gráficos de linha, pontos e muito mais. A função oferece flexibilidade para personalizar os gráficos de diferentes formas.
boxplot()	Essa função cria gráficos de caixa (boxplots), usados para visualizar a distribuição de dados, destacando valores como mediana, quartis e possíveis outliers. Ele facilita a comparação de dispersão e tendências entre diferentes conjuntos de dados.
hist()	Essa função gera histogramas, que são gráficos usados para mostrar a distribuição de frequência de um conjunto de dados numéricos. Ele divide os dados em intervalos (bins) e mostra quantos valores caem em cada intervalo, revelando a forma da distribuição dos dados.

Agora vejamos funções relacionadas ao acesso de diretórios:

FUNÇÕES DE DIRETÓRIOS	DESCRIÇÃO
getwd()	Essa função retorna o caminho do diretório de trabalho atual no ambiente R. Isso é útil para saber onde o R está lendo ou salvando arquivos no momento.
setwd()	Essa função altera o diretório de trabalho atual para o caminho especificado. Ela é usada para definir o diretório onde arquivos serão lidos ou salvos.
write.csv()	Essa função escreve dados de um data frame em um arquivo CSV (separado por vírgulas) no diretório especificado. Ideal para exportar dados de forma padrão.
write.csv2()	Essa função é semelhante à write.csv(), mas usa o ponto e vírgula como separador de colunas, sendo mais comum em países onde a vírgula é o separador decimal.
write.table()	Essa função permite salvar dados em um arquivo com maior controle sobre os separadores e formatos. Oferece mais opções que write.csv() e write.csv2().
install.packages()	Essa função baixa e instala pacotes do repositório CRAN. Os pacotes são conjuntos de funções e dados que estendem as capacidades do R.

98276239072 - FABIANO CARBONELL DA SILVA

library()	Essa função carrega pacotes previamente instalados, tornando suas funções e dados disponíveis no ambiente de trabalho. É necessário chamá-la após instalar o pacote.
head()	Essa função exibe as primeiras linhas de um objeto em R, como data frames ou matrizes. Por padrão, mostra as seis primeiras linhas, sendo útil para inspecionar dados.

Vejamos os tipos de correlação existentes:

TIPOS DE CORRELAÇÃO	DESCRIÇÃO
CORRELAÇÃO POSITIVA FRACA (r ≅ 0)	Os pontos estão dispersos, mostrando uma relação fraca ou quase inexistente entre as variáveis.
CORRELAÇÃO POSITIVA FORTE (0 < r < 1)	Existe uma tendência clara de os pontos subirem em conjunto, indicando que à medida que uma variável aumenta, a outra também aumenta, com uma relação forte, mas não perfeita.
CORRELAÇÃO POSITIVA PERFEITA (r = 1)	Todos os pontos estão alinhados em uma linha reta ascendente, indicando uma correlação perfeita positiva entre as variáveis.
CORRELAÇÃO NEGATIVA FRACA (r ≅ 0)	Os pontos estão dispersos, mostrando uma relação fraca ou quase inexistente, mas com uma leve tendência negativa.
CORRELAÇÃO NEGATIVA FORTE (-1 < r < 0)	Existe uma tendência clara de os pontos descerem juntos, indicando que à medida que uma variável aumenta, a outra diminui, com uma relação forte, mas não perfeita.
CORRELAÇÃO NEGATIVA PERFEITA (r = -1)	Todos os pontos estão alinhados em uma linha reta descendente, indicando uma correlação perfeita negativa entre as variáveis.

Agora vejamos as funções relacionadas a correlação e regressão:

CORRELAÇÃO E REGRESSÃO	DESCRIÇÃO
cor()	Essa função calcula o coeficiente de correlação entre duas variáveis, mostrando a força e a direção do relacionamento linear entre elas. O valor resultante varia de -1 (correlação negativa perfeita) a 1 (correlação positiva perfeita), com 0 indicando ausência de correlação. Pode usar diferentes métodos, como Pearson, Kendall ou Spearman.
lm()	Essa função ajusta um modelo de regressão linear a um conjunto de dados. Ela retorna uma fórmula que descreve a relação entre uma variável dependente (resposta) e uma ou mais variáveis independentes (preditoras). Usada para prever valores ou entender a relação entre variáveis.
predict()	Essa função utiliza um modelo ajustado, como um modelo de regressão linear criado com lm(), para fazer predições com 32.1 Programação - Linguagem Python - Completo base em novos dados. Ela prevê valores para as variáveis dependentes, fornecendo estimativas a partir de valores conhecidos ou novos dados das variáveis independentes.

98276239072 - FABIANO CARBONELL DA SILVA

PACOTES	Um pacote em R é uma coleção de funções, dados, e documentação que estende as capacidades do R base. Cada pacote é desenvolvido para resolver um problema específico ou fornecer uma funcionalidade específica, como análise de dados, visualização gráfica ou manipulação de dados.
BIBLIOTECAS	O termo biblioteca em R refere-se ao local onde os pacotes estão armazenados no sistema. Quando você carrega um pacote usando `library()`, você está acessando o conteúdo do pacote dentro de uma biblioteca onde ele está instalado.

Vejamos agora as categorias de pacotes:

CATEGORIAS	DESCRIÇÃO
BASE	São aqueles que vêm pré-instalados com o núcleo da linguagem R e fornecem a funcionalidade essencial da linguagem. Eles contêm as funções fundamentais necessárias para a operação básica do R, como manipulação de objetos, entrada e saída de dados, gráficos básicos, controle de fluxo (como loops e condicionais), e muito mais.
RECOMMENDED	São aqueles que, embora não façam parte do núcleo básico, são considerados essenciais pela comunidade R para operações comuns e de uso frequente. Eles são instalados automaticamente com a instalação padrão do R e foram escolhidos pelos desenvolvedores para ajudar em tarefas comuns, como manipulação de dados, gráficos avançados, e outras operações úteis em análise estatística e computação científica.
CONTRIBUTED	São aqueles criados e mantidos pela comunidade R. Eles não fazem parte da distribuição padrão do R e precisam ser instalados separadamente por meio de repositórios como CRAN, Github, etc. Eles cobrem uma vasta gama de funcionalidades e são submetidos por desenvolvedores de todo o mundo, permitindo que usuários contribuam com ferramentas específicas para seus domínios.

E agora vejamos os principais pacotes:

PACOTES	DESCRIÇÃO
ggplot2	Um pacote popular para criar visualizações de dados baseadas na gramática dos gráficos. Ele permite construir gráficos sofisticados e customizados a partir de camadas, facilitando a criação de diversos tipos de visualizações.
dplyr	Pacote focado em manipulação de dados. Ele oferece verbos intuitivos, como filter(), select() e mutate(), para facilitar operações sobre data frames de forma rápida e eficiente.
tidyverse	Conjunto de pacotes, como ggplot2, dplyr e tidyr, que compartilham uma filosofia de manipulação de dados baseada em "dados tidy", facilitando a importação, manipulação e visualização de dados.
janitor	Pacote útil para limpeza de dados, com funções que facilitam a normalização de nomes de colunas, remoção de dados duplicados e tabulações de dados, tornando o processo de preparação mais simples.
leafleat	Pacote voltado para a criação de mapas interativos no R. Ele facilita a integração com dados espaciais e permite adicionar camadas, marcadores e estilos personalizados diretamente nos mapas.

98276239072 - FABIANO CARBONELL DA SILVA

APOSTA ESTRATÉGICA

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais.

Eu listei abaixo os pontos com maior probabilidade de cobrança no contexto de **Linguagem R**. Estas são as minhas apostas:

1. Vale revisar os operadores relacionais e lógicos, como **>**, **<**, **==**, **!=** e **&**, que retornam valores booleanos (**TRUE** ou **FALSE**) ao comparar ou combinar expressões.

2. Aposto que haverá perguntas sobre operadores de atribuição, como **<-** e **=**, que são usados para armazenar valores em variáveis, com atenção à diferença entre = (**atribuição**) e == (**igualdade**).

3. Pode aparecer uma questão sobre funções matemáticas no R, como **sqrt()** para calcular a raiz quadrada, **log()** para logaritmos e **abs()** para obter valores absolutos.

4. Aposto em perguntas sobre regras de nomenclatura de variáveis, que devem começar com letras ou pontos seguidos de letras, podem conter números e são **case-sensitive**.

5. Aposto que haverá questões sobre **funções em R**, com destaque para sua sintaxe básica (function()), e como podem aceitar argumentos e retornar valores.

6. Vale revisar o uso de **vetores em R**, uma das estruturas de dados mais fundamentais, que são homogêneos e permitem operações vetorizadas, como soma ou multiplicação entre todos os elementos.

7. Pode cair uma questão sobre a criação de sequências numéricas com a função **seq()**, usada para gerar sequências crescentes ou decrescentes com incrementos definidos.

8. Aposto em questões sobre a função **rep()**, usada para repetir valores ou vetores, criando uma sequência de elementos repetidos conforme o número especificado.

9. Aposto que haverá perguntas sobre a função **sample()**, que gera amostras aleatórias de elementos de um vetor, usada para sorteios ou simulações.

10. Por fim, vale revisar a função **set.seed()**, que garante a reprodutibilidade de resultados ao definir uma semente para amostras aleatórias, algo importante em pesquisas científicas.

98276239072 - FABIANO CARBONELL DA SILVA

QUESTÕES ESTRATÉGICAS

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões.

1. (CESPE / MPO – 2024) Considere o seguinte código R.

```
i <- 0
while (i < 6) {
  i <- i + 1
  if (i == 3) {
    next
  }
  print(i)
}
```

Caso o ambiente e o programa não apresentem nenhum tipo de erro ou exceção, a execução desse código retornará o seguinte resultado.

[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6

Comentários:

No código, o comando **`next`** faz com que a iteração atual do laço seja interrompida e a próxima iteração comece, sem executar o que está abaixo. Logo, quando **`i == 3`**, o valor não será impresso.

Vamos detalhar um pouco mais: o código começa com **`i <- 0`**, ou seja, a variável **`i`** é inicializada com o valor **`0`**; o loop **`while`** irá rodar enquanto a condição **`i < 6`** for verdadeira. Em cada iteração, o valor de **`i`** será incrementado em **`1`**; dentro do loop, existe uma condição **`if (i == 3)`**, que verifica se o valor de **`i`** é igual a **`3`**; se **`i`** for igual a **`3`**, a instrução **`next`** é executada. O **`next`** faz com que o loop pule diretamente para a próxima iteração, ou seja, nada após o **`next`** dentro dessa iteração será executado (incluindo o **`print(i)`**); A função **`print(i)`** será chamada em todas as iterações, exceto quando **`i`** for **`3`**. O resultado correto da execução seria:

[1] 1
[1] 2
[1] 4
[1] 5

98276239072 - FABIANO CARBONELL DA SILVA

[1] 6

Gabarito: Errado

2. (CESPE / MPO – 2024) Em R, o sinal de cerquilha (#), no início de uma linha, transforma essa linha em comentário.

Comentários:

Em R, o sinal de cerquilha **(#)** é usado para indicar que o restante da linha é um comentário, que será ignorado pelo interpretador durante a execução do código. Isso permite incluir anotações explicativas sem afetar o comportamento do programa.

Gabarito: Correto

3. (CESPE / MPO – 2024) Considere o código a seguir.

int x: 5;

A execução do código precedente atribuirá o valor 5 à variável x.

Comentários:

Em R, a sintaxe correta para atribuir o valor 5 à variável **x** seria **x <- 5** ou **x = 5**. O código apresentado (**int x: 5;**) não é válido em R, pois utiliza uma sintaxe incorreta e um tipo de declaração de variável que não existe nessa linguagem.

Gabarito: Errado

4. (CESPE / DATAPREV – 2023) Na linguagem R, admite-se que o nome de uma variável seja iniciado com uma letra ou com um número.

Comentários:

Na linguagem R, o nome de uma variável deve começar com uma letra ou um ponto (desde que o ponto não seja seguido por um número). Nomes de variáveis não podem começar com números.

Gabarito: Errado

5. (CESPE / DATAPREV – 2023) O resultado da execução do código R a seguir será 2.

```
x <- 10
y = 12
z = x+y
print(z)
```


98276239072 - FABIANO CARBONELL DA SILVA

Comentários:

 # Atribuição de valores:

x <- 10: A variável x recebe o valor 10. Neste caso, usamos o operador de atribuição <-, que é equivalente a =, mas é considerado uma boa prática em R.
y = 12: A variável y recebe o valor 12. Aqui usamos o operador =, que também funciona para atribuições.

 # Soma de x e y:

z = x + y: A variável z recebe o valor da soma de x e y. Como x é 10 e y é 12, o resultado da soma será 10 + 12, que é 22.

 # Impressão do valor de z:

print(z): A função print() exibe o valor da variável z, que, após a soma, é 22. Logo, o resultado não é 2.

Gabarito: Correto

6. (CESPE / DATAPREV – 2023) Leafleat é uma biblioteca R para criar e manipular uma hierarquia de classes ou árvore de herança, mostrando os relacionamentos entre classes na programação orientada a objetos.

Comentários:

Leaflet é uma biblioteca R utilizada para criar mapas interativos, e não para manipular hierarquias de classes ou árvores de herança. Para representar relacionamentos entre classes na programação orientada a objetos, seriam utilizadas outras ferramentas, como diagramas de classe ou bibliotecas específicas para esse propósito, como DiagrammeR.

Gabarito: Errado

7. (CESPE / SEFIN de FORTALEZA-CE – 2023) Em relação ao software livre R, versão 4.3, julgue o item subsequente, considerando que todas as bibliotecas necessárias foram corretamente instaladas e disponibilizadas.

Considere-se que, após o código a seguir, tenha sido inserida uma quarta linha com um comando capaz de criar uma regressão linear multivariada para uma variável de nome model. Considere-se, ainda, que o código final tenha sido executado com sucesso.

```
x <- c(2, 4, 6, 8)
y <- c(4, 8, 12, 16)
z <- c(3, 5, 7, 9)
```

Nesse caso, a fim de resumir os dados presentes nos vetores x, y e z em apenas um vetor, de modo a mostrar, por exemplo, o maior valor, o menor valor e a média, por meio de

98276239072 - FABIANO CARBONELL DA SILVA

agrupamento usando valores categóricos, seria correto inserir uma quinta linha no código, com o comando summarize(model).

Comentários:

O comando `summarize(model)` não é correto nesse contexto. Para resumir os dados de vetores como `x`, `y` e `z` em R, funções como `summary()` ou agregações manuais, como `min()`, `max()`, e `mean()`, seriam mais apropriadas. Além disso, a função `summarize()` é usada no pacote `dplyr` para resumir dados em dataframes, e não para resumir diretamente vetores simples ou modelos de regressão linear.

Gabarito: Errado

8. (CESPE / SEFIN de FORTALEZA-CE – 2023) Em relação ao software livre R, versão 4.3, julgue o item subsequente, considerando que todas as bibliotecas necessárias foram corretamente instaladas e disponibilizadas.

O código seq(3, 22, length=6) apresentará, ao ser executado, o resultado a seguir.

```
[1]   3   4   5   6   7   8
```

Comentários:

O comando `seq(3, 22, length=6)` gera uma sequência de 6 números igualmente espaçados entre 3 e 22, e não uma sequência de números inteiros consecutivos como mostrado no item. Logo, o resultado correto seria:

```
[1]  3.0   7.8   12.6   17.4   22.0
```

O valor de `length = 6` define o número de elementos na sequência, que são distribuídos uniformemente entre 3 e 22.

Gabarito: Errado

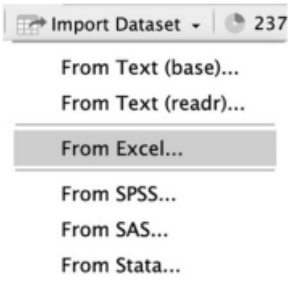
9. (CESPE / SEFIN DE FORTALEZA-CE – 2023) A seguir, são apresentados dados listados no arquivo sefin.csv, que se encontra gravado no diretório /home/sec/ e cujo código foi desenvolvido em linguagem R versão 4.3.

id; nome; cartografia; tipo
3; Quarup; 48-110-130; Médio
4;Mirafiori; 48-110-172; Superior
5; Scopa Residenza; 48-115-117; Superior
6; Luciano Magalhães; 48-26-354; Médio
7; San Marino; 48-26-371; Médio
8; Copenhague; 48-26-393; Proletário
9; Montreal I; 48-26-448; Médio
10; Liberty Place; 48-36-290; Médio
11; Tia Joana II;50-213-124; Proletário

98276239072 - FABIANO CARBONELL DA SILVA

12; Tia Joana III; 50-239-123; Econômico

Embora o RStudio 2023 permita importar dados diretamente de arquivos Excel a partir da opção Import Dataset, em destaque na figura a seguir, por questão de segurança, somente é possível obter dados de arquivos gravados localmente no servidor que esteja executando o R ou na máquina do usuário, ou seja, não é possível importar dados por meio de URL.



Comentários:

No R, é possível importar dados por meio de URLs utilizando funções como **`read.csv()`** para arquivos CSV e **`read_excel()`** para arquivos MS-Excel (se o pacote **`readxl`** estiver instalado), desde que a URL forneça acesso direto ao arquivo. Embora a interface gráfica do RStudio foque na importação local, o R permite carregar dados de arquivos remotos, desde que as permissões e formatos sejam compatíveis.

Gabarito: Errado

10. (CESPE / SEFIN de FORTALEZA-CE – 2023) A seguir, são apresentados dados listados no arquivo sefin.csv, que se encontra gravado no diretório /home/sec/ e cujo código foi desenvolvido em linguagem R versão 4.3.

id; nome; cartografia; tipo
3; Quarup; 48-110-130; Médio
4;Mirafiori; 48-110-172; Superior
5; Scopa Residenza; 48-115-117; Superior
6; Luciano Magalhães; 48-26-354; Médio
7; San Marino; 48-26-371; Médio
8; Copenhague; 48-26-393; Proletário
9; Montreal I; 48-26-448; Médio
10; Liberty Place; 48-36-290; Médio
11; Tia Joana II;50-213-124; Proletário
12; Tia Joana III; 50-239-123; Econômico

De acordo com o código a seguir, a variável sefin receberá corretamente os dados importados do arquivo sefin.csv.

```
library(readr)
endereco = “/home/sec/sefin.csv”
sefin <- read.csv(endereco, sep=“;”)
```


98276239072 - FABIANO CARBONELL DA SILVA

```
head(sefin)
```

Comentários:

O código apresentado importará corretamente o arquivo **sefin.csv** usando a função **read.csv()**, especificando o separador **sep=";"**, que é adequado ao formato do arquivo. A variável **sefin** conterá os dados importados, e o comando **head(sefin)** exibirá as primeiras seis linhas do conjunto de dados.

Gabarito: Correto

11. (CESPE / SEFIN de FORTALEZA-CE – 2023) A seguir, são apresentados dados listados no arquivo sefin.csv, que se encontra gravado no diretório /home/sec/ e cujo código foi desenvolvido em linguagem R versão 4.3.

id; nome; cartografia; tipo
3; Quarup; 48-110-130; Médio
4;Mirafiori; 48-110-172; Superior
5; Scopa Residenza; 48-115-117; Superior
6; Luciano Magalhães; 48-26-354; Médio
7; San Marino; 48-26-371; Médio
8; Copenhague; 48-26-393; Proletário
9; Montreal I; 48-26-448; Médio
10; Liberty Place; 48-36-290; Médio
11; Tia Joana II;50-213-124; Proletário
12; Tia Joana III; 50-239-123; Econômico

O código

```
library(readr)
endereco = “/home/sec/sefin.csv”
sefin <- read.csv(endereco, sep=“;”)
head(sefin,2,3)
```

produzirá, ao ser executado, o resultado a seguir.

```
1 nome
2 Quarup
3 Mirafiori
4 Scopa Residenza
```

Comentários:

O código contém um erro na função **head()**. O correto seria **head(sefin, 2)** para exibir as duas primeiras linhas do dataset, já que **head()** só aceita dois parâmetros: o conjunto de dados e o número de linhas. Além disso, o resultado apresentado está incorreto, pois a função **head()** exibe linhas completas, incluindo todas as colunas, não apenas a coluna "nome". O resultado esperado incluiria as primeiras duas linhas completas do dataset.

98276239072 - FABIANO CARBONELL DA SILVA

Gabarito: Errado

12. (CESPE / FUB – 2022) Com respeito aos comandos de repetição da linguagem R, julgue o item que se segue.

Com respeito ao código mostrado na figura a seguir, é correto afirmar que a variável x resultante da aplicação do loop repeat é igual ao vetor c(4,3,2,1,0).

Comentários:

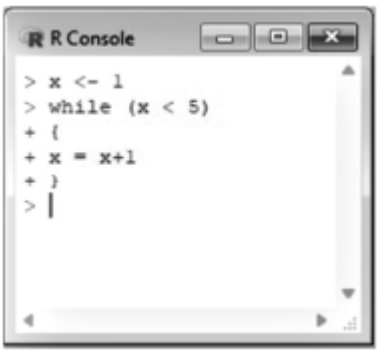
A variável **x** resultante da execução do código não será o vetor **c(4, 3, 2, 1, 0)**, mas sim o valor numérico **-1**. O loop **repeat** apenas imprime os valores de **x** conforme ele é decrementado, mas não armazena esses valores em um vetor. Após o loop terminar, o valor final de **x** é **-1**, pois o laço é interrompido quando **x** se torna negativo. Detalhando...

Inicialmente, **x** é **4**, então o loop começa e **4** é impresso; depois, **x** é decrementado para **3**, que também é impresso; isso continua com **x** tomando os valores **2**, **1** e **0**, sendo impressos a cada iteração; quando x atinge **-1** após o último decremento, a condição **if (x < 0)** é verdadeira, o break é acionado, e o loop é interrompido.

Gabarito: Errado

13. (CESPE / FUB – 2022) Ao final do loop while mostrado na figura a seguir, o valor da variável x será igual a 4.

98276239072 - FABIANO CARBONELL DA SILVA



Comentários:

No código apresentado, a variável **x** é incrementada em cada iteração do laço **while** até que **x** seja igual ou maior que **5**. Logo, ao final do loop, **x** terá o valor de **5**, pois é o primeiro número que não satisfaz a condição **x < 5**.

Gabarito: Errado

14. (CESPE / POLÍCIA FEDERAL – 2021) Com relação a conceitos de programação Python e R, julgue o item que se segue.

O resultado do código R seguinte será "12".

```
f<- function(x) {
        g <- function(y) {
                y + z
        }
        z <- 4
        x + g(x)
}
z <- 10
f (4)
```

Comentários:

Essa questão foi bastante polêmica, portanto vamos detalhá-la. Vejamos:

Definição da função f(x): a função f recebe um argumento x; dentro de f, é definida outra função g(y) que recebe um argumento y; a função g retorna o valor da soma de y mais z.

Função g(y): a função g retorna o valor de y + z; a variável z não é passada como argumento para g. Isso significa que o R tentará encontrar o valor de z no ambiente onde g está sendo executada. A função g(y) está aninhada dentro da função f(x). Isso significa que o R tentará encontrar z no escopo da função f.

Definição de z dentro de f(x): dentro de f, existe uma variável z localmente definida com o valor 4. Como essa variável z está definida no mesmo escopo da função g(y), ela será usada pela função g ao ser chamada dentro de f.

98276239072 - FABIANO CARBONELL DA SILVA

Execução de x + g(x): depois de definir z <- 4, a função f retorna a soma de x com o resultado de g(x). Quando g(x) é chamada, o valor de x (passado como argumento para f) é utilizado como argumento de y em g. Dentro de g, o cálculo realizado é y + z, onde y é o valor de x (ou seja, 4) e z é o valor local definido dentro de f (ou seja, 4).

Variável global z: fora da função, existe uma variável global z com o valor 10, mas essa variável não interfere na execução de g(x) porque dentro de f, uma variável local z foi definida com valor 4, e essa é a que será utilizada na função.

Agora vamos ver como o código efetivamente foi executado: quando chamado f(4), o valor de x será 4. Correto? Dentro de f, a função g(x) será chamada. Neste caso, g utilizará y = 4 e somará com z = 4 (a variável local z definida dentro de f). Logo, a soma dentro de g será 4 + 4 = 8. Por fim, a função f retornará x + g(x), ou seja, 4 + 8 = 12.

Bacana! Agora vem a polêmica...

O enunciado diz que o resultado do código R seguinte será "12". Ocorre que 12 é diferente de "12": 12 é um número e "12" é um texto. O resultado do código é 12 e, não, "12".

O gabarito preliminar foi correto, mas alguns alunos entraram com recurso argumentando correto seria 12 em vez de "12" e a banca deferiu.

Vários alunos ficaram chateados porque entenderam que as aspas, nesse contexto, estavam apenas enfatizando o valor 12. Enfim, foi uma confusão danada!

Gabarito: Errado

15. (CESPE / POLÍCIA FEDERAL – 2021) Considere o programa a seguir, escrito em R.

```
x <- TRUE
y <- FALSE
print (xy)
```

Após a execução do programa, será obtido o seguinte resultado.

```
[1] FALSE
```

Comentários:

Olha a pegadinha do examinador! Ele declara uma variável x e inicializa com valor TRUE, declara uma variável y inicializa com valor FALSE e pede para imprimir uma xy, que sequer foi declarada e muito menos inicializada. Logo, resultará em erro, porque essa variável simplesmente não existe. Vejam só:

98276239072 - FABIANO CARBONELL DA SILVA

```
Console   Terminal ×   Background Jobs ×
R 4.4.1 · ~/
> x <- TRUE
>
> y <- FALSE
>
>
> print (xy)
Erro: objeto 'xy' não encontrado
>
```

De forma análoga, imagine que eu declare as seguintes variáveis:

di <- 2
e <- 4
go <- 6

E eu tente execute: print(diego), resultará em erro porque não existe nenhuma variável diego. Para essa questão ser verdadeira, o código poderia ser print(x&y) em vez de print(xy). Como o operador AND só retorna TRUE quando ambos os operandos são TRUE, o resultado seria realmente [1] FALSE. No entanto, esse não foi o caso da questão original.

- **Gabarito:** Errado

98276239072 - FABIANO CARBONELL DA SILVA

QUESTIONÁRIO DE REVISÃO E APERFEIÇOAMENTO

A ideia do questionário é elevar o nível da sua compreensão no assunto e, ao mesmo tempo, proporcionar uma outra forma de revisão de pontos importantes do conteúdo, a partir de perguntas que exigem respostas subjetivas.

São questões um pouco mais desafiadoras, porque a redação de seu enunciado não ajuda na sua resolução, como ocorre nas clássicas questões objetivas.

O objetivo é que você realize uma autoexplicação mental de alguns pontos do conteúdo, para consolidar melhor o que aprendeu ;)

Além disso, as questões objetivas, em regra, abordam pontos isolados de um dado assunto. Assim, ao resolver várias questões objetivas, o candidato acaba memorizando pontos isolados do conteúdo, mas muitas vezes acaba não entendendo como esses pontos se conectam.

Assim, no questionário, buscaremos trazer também situações que ajudem você a conectar melhor os diversos pontos do conteúdo, na medida do possível.

É importante frisar que não estamos adentrando em um nível de profundidade maior que o exigido na sua prova, mas apenas permitindo que você compreenda melhor o assunto de modo a facilitar a resolução de questões objetivas típicas de concursos, ok?

Nosso compromisso é proporcionar a você uma revisão de alto nível! Vamos ao nosso questionário:

Perguntas

1. **O que é a linguagem R?**
2. **Por que o R é preferido ao MS-Excel para análise estatística?**
3. **O que é o RStudio?**
4. **Qual é a função do Console no RStudio?**
5. **O que são operadores aritméticos na linguagem R?**
6. **Qual a precedência dos operadores em R?**
7. **Para que servem os operadores relacionais no R?**
8. **O que fazem os operadores lógicos no R?**
9. **Qual a diferença entre o operador = e o operador == no R?**

98276239072 - FABIANO CARBONELL DA SILVA

10. O que faz a função sqrt() no R?
11. Como funciona a função log() no R?
12. O que representa o tipo de dado "numeric" no R?
13. Como o R trata a diferenciação de maiúsculas e minúsculas em variáveis?
14. Qual é o escopo de uma variável global em R?
15. Qual a característica principal de uma variável local?
16. O que faz a função print() no R?
17. Como se define uma função no R?
18. Para que serve a função help() no R?
19. O que é a função ls() no R?
20. O que faz a função xor() no R?
21. Qual a importância dos comentários no R?
22. O que são vetores no R?
23. O que a função c() faz no R?
24. Como a função mean() é utilizada no R?
25. Para que serve a função sum() no R?
26. Como acessar o primeiro elemento de um vetor em R?
27. O que faz o comando "notas[-2]" em um vetor no R?
28. Como funciona a função seq() no R?
29. Como se cria um vetor com a função rep() no R?
30. Para que serve a função sample() no R?
31. O que faz a função set.seed() no R?
32. Como a função rev() opera no R?
33. Qual a função da função sort() no R?

98276239072 - FABIANO CARBONELL DA SILVA

34. O que são matrizes no R?
35. Como a função cbind() é usada no R?
36. O que faz a função rbind() no R?
37. Como verificar as dimensões de uma matriz no R?
38. Como acessar o elemento de uma matriz na linha 3, coluna 2 no R?
39. Como remover uma linha de uma matriz no R?
40. O que faz a função matrix() no R?
41. Como se define uma matriz identidade no R?
42. O que faz a função t() no R?
43. Qual a regra para multiplicação de matrizes no R?
44. Como se obtém a diagonal principal de uma matriz no R?
45. Como remover uma coluna de uma matriz no R?
46. O que é uma matriz nula no R?
47. Como funciona a função diag() para criar uma matriz diagonal?
48. Qual a diferença entre a multiplicação escalar e a multiplicação matricial no R?
49. O que ocorre ao tentar multiplicar matrizes de dimensões incompatíveis no R?
50. Para que serve a função %*% no R?
51. Como somar duas matrizes no R?
52. Qual é o conceito de matriz transposta?
53. O que faz a função colSums() no R?
54. O que faz a função rowSums() no R?
55. Como a função apply() opera em uma matriz no R?
56. O que acontece ao tentar acessar um índice inexistente em uma matriz?
57. Para que serve a função length() no R?

98276239072 - FABIANO CARBONELL DA SILVA

58. Qual é a função da função typeof() no R?

59. Como se verifica o tipo de um objeto no R?

60. O que acontece ao tentar realizar uma operação aritmética entre vetores de comprimentos diferentes?

98276239072 - FABIANO CARBONELL DA SILVA

Perguntas com Respostas

1. O que é a linguagem R?

R é uma linguagem de programação especializada em estatísticas, muito usada em análise de dados e aprendizado de máquina, oferecendo uma vasta gama de funções matemáticas e ferramentas para grandes volumes de dados.

2. Por que o R é preferido ao MS-Excel para análise estatística?

R é preferido por sua capacidade de lidar com grandes volumes de dados, além de ser um software livre, mais flexível e colaborativo, não se limitando como o Excel em termos de linhas e funções avançadas.

3. O que é o RStudio?

RStudio é um Ambiente de Desenvolvimento Integrado (IDE) para o R, que facilita a análise de dados, construção de relatórios e oferece uma seção de ajuda detalhada para suporte ao desenvolvimento de scripts.

4. Qual é a função do Console no RStudio?

O Console no RStudio é o local onde o usuário interage diretamente com o R, inserindo comandos e visualizando resultados instantaneamente através de uma interface de linha de comando.

5. O que são operadores aritméticos na linguagem R?

Operadores aritméticos no R são usados para realizar operações matemáticas básicas, como adição (+), subtração (-), multiplicação (*), divisão (/) e exponenciação (^ ou **).

6. Qual a precedência dos operadores em R?

A precedência dos operadores em R segue a ordem: Exponenciação > Multiplicação/Divisão > Soma/Subtração, com operações resolvidas da esquerda para a direita, exceto a exponenciação, que é da direita para a esquerda.

7. Para que servem os operadores relacionais no R?

Operadores relacionais comparam dois valores, retornando um valor lógico (TRUE ou FALSE), como maior que (>) ou igual (==), usados em verificações e comparações de dados.

8. O que fazem os operadores lógicos no R?

Os operadores lógicos combinam ou invertem expressões lógicas, retornando TRUE ou FALSE. Exemplos incluem o E (&), OU (|), NÃO (!) e suas variações para vetores.

98276239072 - FABIANO CARBONELL DA SILVA

9. Qual a diferença entre o operador = e o operador == no R?

O operador = é usado para atribuir valores a variáveis, enquanto o operador == é usado para comparar dois valores, verificando se são iguais.

10. O que faz a função sqrt() no R?

A função sqrt() calcula a raiz quadrada de um número, retornando o valor resultante.

11. Como funciona a função log() no R?

A função log() no R calcula o logaritmo natural de um número, retornando o valor em base natural (e).

12. O que representa o tipo de dado "numeric" no R?

O tipo "numeric" representa números em ponto flutuante ou decimais, usados amplamente para cálculos e operações numéricas.

13. Como o R trata a diferenciação de maiúsculas e minúsculas em variáveis?

O R é case-sensitive, diferenciando maiúsculas e minúsculas, o que significa que "Variavel" e "variavel" são considerados objetos distintos.

14. Qual é o escopo de uma variável global em R?

Variáveis globais são acessíveis e modificáveis em qualquer parte do programa após sua definição, inclusive dentro de funções, a menos que uma variável local com o mesmo nome seja criada.

15. Qual a característica principal de uma variável local?

Variáveis locais são definidas dentro de uma função e só podem ser acessadas dentro dessa função, deixando de existir após sua execução.

16. O que faz a função print() no R?

A função print() exibe o valor de variáveis ou expressões no console, útil para scripts que necessitam exibir resultados de operações intermediárias.

17. Como se define uma função no R?

Funções são blocos de código reutilizáveis que recebem argumentos, realizam uma tarefa e retornam um resultado. São definidas com "function()" e podem ter argumentos opcionais.

18. Para que serve a função help() no R?

98276239072 - FABIANO CARBONELL DA SILVA

A função help() exibe a documentação de funções e objetos no R, explicando sua sintaxe, exemplos e informações detalhadas de uso.

19. O que é a função ls() no R?

A função ls() lista todos os objetos presentes no ambiente de trabalho atual, facilitando o gerenciamento de variáveis e funções criadas durante a sessão.

20. O que faz a função xor() no R?

A função xor() executa a operação lógica "OU exclusivo", retornando TRUE se um valor for verdadeiro e o outro falso, ou FALSE em caso contrário.

21. Qual a importância dos comentários no R?

Comentários, feitos com "#", são usados para explicar partes do código, tornando-o mais compreensível, sem interferir na execução do programa.

22. O que são vetores no R?

Vetores são estruturas de dados unidimensionais que armazenam múltiplos valores do mesmo tipo, como números, caracteres ou lógicos.

23. O que a função c() faz no R?

A função c() concatena valores para formar vetores, permitindo agrupar dados de maneira eficiente em uma única estrutura de dados.

24. Como a função mean() é utilizada no R?

A função mean() calcula a média dos elementos de um vetor numérico, retornando o valor médio dos dados contidos no vetor.

25. Para que serve a função sum() no R?

A função sum() calcula a soma de todos os elementos de um vetor numérico, retornando o valor total da adição.

26. Como acessar o primeiro elemento de um vetor em R?

Para acessar o primeiro elemento de um vetor no R, utiliza-se o índice 1 entre colchetes. Exemplo: vetor[1].

27. O que faz o comando "notas[-2]" em um vetor no R?

O comando notas[-2] acessa todos os elementos do vetor exceto aquele localizado na posição de índice 2, ou seja, remove o segundo elemento.

98276239072 - FABIANO CARBONELL DA SILVA

28. Como funciona a função seq() no R?

A função seq() gera uma sequência numérica, definindo o valor inicial, final e o incremento (ou decremento) entre os elementos da sequência.

29. Como se cria um vetor com a função rep() no R?

A função rep() repete um valor ou vetor por um número definido de vezes, criando um vetor de elementos repetidos.

30. Para que serve a função sample() no R?

A função sample() gera amostras aleatórias de um conjunto de elementos, podendo definir se a amostragem é com ou sem reposição.

31. O que faz a função set.seed() no R?

A função set.seed() garante a reprodutibilidade de amostras aleatórias, definindo uma semente para a geração de números aleatórios.

32. Como a função rev() opera no R?

A função rev() inverte a ordem dos elementos de um vetor, retornando um novo vetor com a sequência invertida.

33. Qual a função da função sort() no R?

A função sort() ordena os elementos de um vetor em ordem crescente ou decrescente, podendo também definir o tratamento de valores ausentes (NA).

34. O que são matrizes no R?

Matrizes são estruturas de dados bidimensionais compostas por linhas e colunas, onde todos os elementos são do mesmo tipo.

35. Como a função cbind() é usada no R?

A função cbind() combina vetores por colunas, criando uma matriz onde cada vetor se torna uma coluna.

36. O que faz a função rbind() no R?

A função rbind() combina vetores por linhas, criando uma matriz onde cada vetor se torna uma linha.

37. Como verificar as dimensões de uma matriz no R?

98276239072 - FABIANO CARBONELL DA SILVA

A função dim() retorna as dimensões de uma matriz, exibindo o número de linhas e colunas da estrutura.

38. Como acessar o elemento de uma matriz na linha 3, coluna 2 no R?

Para acessar um elemento específico de uma matriz, utiliza-se a sintaxe matriz[3, 2], onde o primeiro número é a linha e o segundo a coluna.

39. Como remover uma linha de uma matriz no R?

Para remover uma linha de uma matriz, usa-se a referência negativa do índice da linha, como em matriz[-2,].

40. O que faz a função matrix() no R?

A função matrix() cria uma matriz a partir de um vetor de dados, especificando o número de linhas e colunas da matriz.

41. Como se define uma matriz identidade no R?

Uma matriz identidade é uma matriz quadrada onde os elementos da diagonal principal são iguais a 1, e os outros elementos são 0.

42. O que faz a função t() no R?

A função t() realiza a transposição de uma matriz, trocando suas linhas por colunas e vice-versa.

43. Qual a regra para multiplicação de matrizes no R?

A multiplicação matricial só é possível quando o número de colunas da primeira matriz é igual ao número de linhas da segunda matriz.

44. Como se obtém a diagonal principal de uma matriz no R?

A função diag() extrai ou substitui os elementos da diagonal principal de uma matriz, retornando um vetor com esses valores.

45. Como remover uma coluna de uma matriz no R?

Para remover uma coluna de uma matriz, usa-se a referência negativa do índice da coluna, como em matriz[, -1].

46. O que é uma matriz nula no R?

Uma matriz nula é uma matriz em que todos os seus elementos são iguais a zero, independentemente de sua dimensão.

47. Como funciona a função diag() para criar uma matriz diagonal?

98276239072 - FABIANO CARBONELL DA SILVA

A função diag() pode ser usada para criar uma matriz diagonal onde os valores fornecidos serão colocados na diagonal principal e os demais elementos serão zero.

48. Qual a diferença entre a multiplicação escalar e a multiplicação matricial no R?

A multiplicação escalar multiplica cada elemento da matriz por um número, enquanto a multiplicação matricial segue regras específicas entre linhas e colunas de matrizes.

49. O que ocorre ao tentar multiplicar matrizes de dimensões incompatíveis no R?

Se as dimensões das matrizes não forem compatíveis para multiplicação, o R retornará um erro, já que o número de colunas da primeira matriz deve ser igual ao número de linhas da segunda.

50. Para que serve a função %*% no R?

A função %*% realiza a multiplicação matricial no R, multiplicando uma matriz pelas colunas e linhas correspondentes de outra.

51. Como somar duas matrizes no R?

Para somar duas matrizes no R, ambas devem ter as mesmas dimensões. A soma é feita somando os elementos de mesma posição de cada matriz.

52. Qual é o conceito de matriz transposta?

Uma matriz transposta é obtida ao trocar as linhas de uma matriz por colunas, alterando suas dimensões e organização dos elementos.

53. O que faz a função colSums() no R?

A função colSums() calcula a soma dos elementos de cada coluna de uma matriz, retornando um vetor com as somas das colunas.

54. O que faz a função rowSums() no R?

A função rowSums() calcula a soma dos elementos de cada linha de uma matriz, retornando um vetor com as somas das linhas.

55. Como a função apply() opera em uma matriz no R?

A função apply() aplica uma função a cada linha ou coluna de uma matriz, permitindo, por exemplo, somar ou calcular médias de elementos de forma eficiente.

56. O que acontece ao tentar acessar um índice inexistente em uma matriz?

Se um índice inexistente for acessado em uma matriz, o R retornará um valor "NA", indicando que o índice está fora dos limites da matriz.

98276239072 - FABIANO CARBONELL DA SILVA

57. Para que serve a função length() no R?

A função length() retorna o número total de elementos de um vetor ou o comprimento de outros objetos, como matrizes e listas.

58. Qual é a função da função typeof() no R?

A função typeof() retorna o tipo interno de um objeto em R, fornecendo detalhes mais técnicos sobre sua estrutura, como "double", "integer" ou "character".

59. Como se verifica o tipo de um objeto no R?

A função class() verifica e retorna a classe (tipo) de um objeto, como "numeric", "character", "logical", entre outros.

60. O que acontece ao tentar realizar uma operação aritmética entre vetores de comprimentos diferentes?

Se os vetores tiverem comprimentos diferentes, o R tenta "reciclar" os elementos do vetor menor. Se a reciclagem não for exata, um aviso será gerado.

98276239072 - FABIANO CARBONELL DA SILVA

LISTA DE QUESTÕES ESTRATÉGICAS

1. **(CESPE / MPO – 2024) Considere o seguinte código R.**

```
i <- 0
while (i < 6) {
  i <- i + 1
  if (i == 3) {
    next
  }
  print(i)
}
```

Caso o ambiente e o programa não apresentem nenhum tipo de erro ou exceção, a execução desse código retornará o seguinte resultado.

[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6

2. **(CESPE / MPO – 2024) Em R, o sinal de cerquilha (#), no início de uma linha, transforma essa linha em comentário.**

3. **(CESPE / MPO – 2024) Considere o código a seguir.**

int x: 5;

A execução do código precedente atribuirá o valor 5 à variável x.

4. **(CESPE / DATAPREV – 2023) Na linguagem R, admite-se que o nome de uma variável seja iniciado com uma letra ou com um número.**

5. **(CESPE / DATAPREV – 2023) O resultado da execução do código R a seguir será 2.**

```
x <- 10
y = 12
z = x+y
print(z)
```

6. **(CESPE / DATAPREV – 2023) Leafleat é uma biblioteca R para criar e manipular uma hierarquia de classes ou árvore de herança, mostrando os relacionamentos entre classes na programação orientada a objetos.**

98276239072 - FABIANO CARBONELL DA SILVA

7. (CESPE / SEFIN de FORTALEZA-CE – 2023) Em relação ao software livre R, versão 4.3, julgue o item subsequente, considerando que todas as bibliotecas necessárias foram corretamente instaladas e disponibilizadas.

Considere-se que, após o código a seguir, tenha sido inserida uma quarta linha com um comando capaz de criar uma regressão linear multivariada para uma variável de nome model. Considere-se, ainda, que o código final tenha sido executado com sucesso.

```
x <- c(2, 4, 6, 8)
y <- c(4, 8, 12, 16)
z <- c(3, 5, 7, 9)
```

Nesse caso, a fim de resumir os dados presentes nos vetores x, y e z em apenas um vetor, de modo a mostrar, por exemplo, o maior valor, o menor valor e a média, por meio de agrupamento usando valores categóricos, seria correto inserir uma quinta linha no código, com o comando summarize(model).

8. (CESPE / SEFIN de FORTALEZA-CE – 2023) Em relação ao software livre R, versão 4.3, julgue o item subsequente, considerando que todas as bibliotecas necessárias foram corretamente instaladas e disponibilizadas.

O código seq(3, 22, length=6) apresentará, ao ser executado, o resultado a seguir.

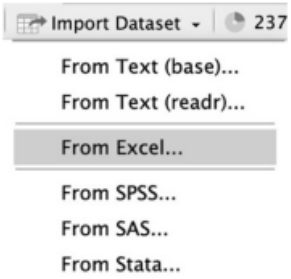
```
[1]  3  4  5  6  7  8
```

9. (CESPE / SEFIN DE FORTALEZA-CE – 2023) A seguir, são apresentados dados listados no arquivo sefin.csv, que se encontra gravado no diretório /home/sec/ e cujo código foi desenvolvido em linguagem R versão 4.3.

id; nome; cartografia; tipo
3; Quarup; 48-110-130; Médio
4;Mirafiori; 48-110-172; Superior
5; Scopa Residenza; 48-115-117; Superior
6; Luciano Magalhães; 48-26-354; Médio
7; San Marino; 48-26-371; Médio
8; Copenhague; 48-26-393; Proletário
9; Montreal I; 48-26-448; Médio
10; Liberty Place; 48-36-290; Médio
11; Tia Joana II;50-213-124; Proletário
12; Tia Joana III; 50-239-123; Econômico

Embora o RStudio 2023 permita importar dados diretamente de arquivos Excel a partir da opção Import Dataset, em destaque na figura a seguir, por questão de segurança, somente é possível obter dados de arquivos gravados localmente no servidor que esteja executando o R ou na máquina do usuário, ou seja, não é possível importar dados por meio de URL.

98276239072 - FABIANO CARBONELL DA SILVA



10. (CESPE / SEFIN de FORTALEZA-CE – 2023) A seguir, são apresentados dados listados no arquivo sefin.csv, que se encontra gravado no diretório /home/sec/ e cujo código foi desenvolvido em linguagem R versão 4.3.

id; nome; cartografia; tipo
3; Quarup; 48-110-130; Médio
4;Mirafiori; 48-110-172; Superior
5; Scopa Residenza; 48-115-117; Superior
6; Luciano Magalhães; 48-26-354; Médio
7; San Marino; 48-26-371; Médio
8; Copenhague; 48-26-393; Proletário
9; Montreal I; 48-26-448; Médio
10; Liberty Place; 48-36-290; Médio
11; Tia Joana II;50-213-124; Proletário
12; Tia Joana III; 50-239-123; Econômico

De acordo com o código a seguir, a variável sefin receberá corretamente os dados importados do arquivo sefin.csv.

```
library(readr)
endereco = “/home/sec/sefin.csv”
sefin <- read.csv(endereco, sep=“;”)
head(sefin)
```

11. (CESPE / SEFIN de FORTALEZA-CE – 2023) A seguir, são apresentados dados listados no arquivo sefin.csv, que se encontra gravado no diretório /home/sec/ e cujo código foi desenvolvido em linguagem R versão 4.3.

id; nome; cartografia; tipo
3; Quarup; 48-110-130; Médio
4;Mirafiori; 48-110-172; Superior
5; Scopa Residenza; 48-115-117; Superior
6; Luciano Magalhães; 48-26-354; Médio
7; San Marino; 48-26-371; Médio
8; Copenhague; 48-26-393; Proletário
9; Montreal I; 48-26-448; Médio
10; Liberty Place; 48-36-290; Médio
11; Tia Joana II;50-213-124; Proletário
12; Tia Joana III; 50-239-123; Econômico

98276239072 - FABIANO CARBONELL DA SILVA

O código

```
library(readr)
endereco = “/home/sec/sefin.csv”
sefin <- read.csv(endereco, sep=“;”)
head(sefin,2,3)
```

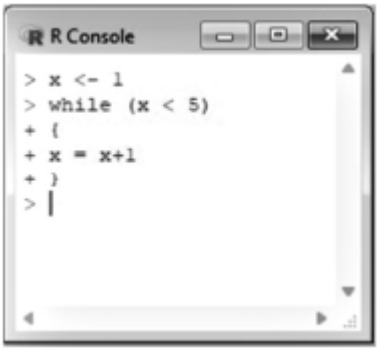
produzirá, ao ser executado, o resultado a seguir.

```
1 nome
2 Quarup
3 Mirafiori
4 Scopa Residenza
```

12. (CESPE / FUB – 2022) Com respeito aos comandos de repetição da linguagem R, julgue o item que se segue.

Com respeito ao código mostrado na figura a seguir, é correto afirmar que a variável x resultante da aplicação do loop repeat é igual ao vetor c(4,3,2,1,0).

13. (CESPE / FUB – 2022) Ao final do loop while mostrado na figura a seguir, o valor da variável x será igual a 4.



98276239072 - FABIANO CARBONELL DA SILVA

14. (CESPE / POLÍCIA FEDERAL – 2021) Com relação a conceitos de programação Python e R, julgue o item que se segue.

O resultado do código R seguinte será "12".

```
f<- function(x) {
        g <- function(y) {
                y + z
        }
        z <- 4
        x + g(x)
}
z <- 10
f (4)
```

15. (CESPE / POLÍCIA FEDERAL – 2021) Considere o programa a seguir, escrito em R.

```
x <- TRUE
y <- FALSE
print (xy)
```

Após a execução do programa, será obtido o seguinte resultado.

```
[1] FALSE
```


98276239072 - FABIANO CARBONELL DA SILVA

GABARITO

1. ERRADO
2. CORRETO
3. ERRADO
4. ERRADO
5. CORRETO
6. ERRADO
7. ERRADO
8. ERRADO
9. ERRADO
10. CORRETO
11. ERRADO
12. ERRADO
13. ERRADO
14. ERRADO
15. ERRADO

98276239072 - FABIANO CARBONELL DA SILVA

Referências Bibliográficas

R CORE TEAM. R Documentation. Disponível em: https://cran.r-project.org/manuals.html. Acesso em: 24 set. 2024.

98276239072 - FABIANO CARBONELL DA SILVA

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.

Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.